FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

IEC 61499 Compliant Web-based IDE for Function Block Design

Pedro da Cunha Teixeira e Faro Beirão

WORKING VERSION

Mestrado em Engenharia Informática e Computação

Supervisor: Prof. Rui Pinto

May 10, 2026

Contents

List of Figures

List of Acronyms

CPPS Cyber-Physical Production Systems

FBD Function Block Diagram

ICS Industrial Control Systems

IDE Integrated Development Environment

PLC Programmable Logic Controller

RTE Runtime Environment

SOA Service-Oriented Architectures

UI User Interface

Chapter 1

Introduction

1.1 Context

Cyber-Physical Production Systems (CPPS) [6] are complex systems that integrate physical processes with digital technologies to optimise production processes. Several software engineering approaches and technologies are utilised in the development and operation of CPPS. One of such approaches is Function Block Diagram (FBD) [2], which are graphical programming languages used in industrial automation and control systems. FBD uses graphical function blocks to represent the logic and control of a system. The function blocks are connected in a network to form a control program. One of the benefits of FBD is that it provides a visual representation of the control logic, which can be easier to understand and debug than traditional textual programming languages. FBD is also modular, which means that function blocks can be reused in multiple programs, reducing development time and improving code maintainability. Also, FBD is often used in conjunction with Programmable Logic Controllers (PLCs) to control machinery and processes in manufacturing and other industrial applications [7, 8]. FBD programs are not as simple to create and edit as traditional text-based languages since they require the use of specialised software tools that allow for the manipulation of function blocks [16].

In industrial automation, it is common to use the *IEC 61499* standard, which defines an event-driven FBD approach for designing control software [11]. Unlike traditional PLC programming languages, *IEC 61499* emphasises modularity, reusability, and explicit separation of events and data, allowing developers to design complex distributed systems in a scalable and maintainable way. Function blocks define both algorithms and interfaces for inputs, outputs, and events, making them ideal for CPPS development [3]. This approach is particularly suitable for distributed automation systems, where control logic must respond to events generated by multiple devices or sensors in real time. An example of such a function block interface is shown in Figure 1.1, highlighting the event inputs, outputs, and data connections that enable flexible interaction between system components.

While Runtime Environments (RTEs) enable the execution of function blocks compliant with the standard, Integrated Development Environments (IDEs) are used to design, create, and deploy

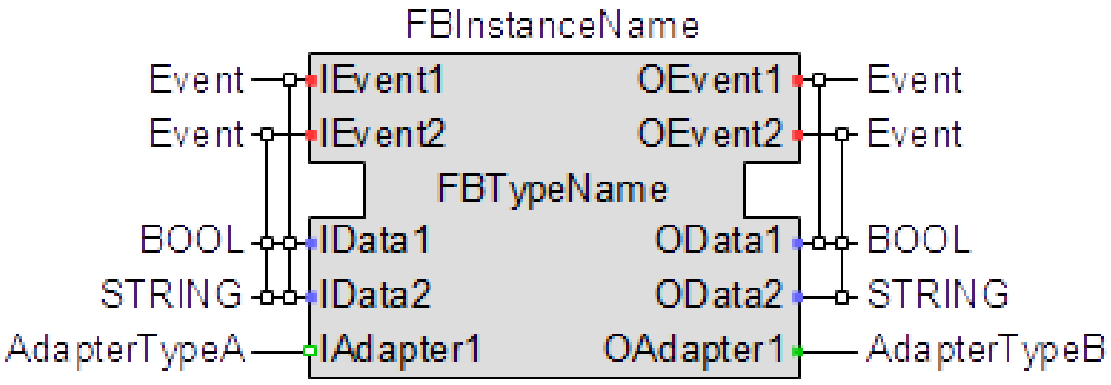

Figure 1.1: *IEC 61499* function block interface [4]

function block pipelines to CPPS. These IDEs offer an User Interface (UI) with features and tools that facilitate the development, testing, and deployment of *IEC 61499*-compliant control systems. They are graphical editors, like shown in Figure 1.2, for creating function block diagrams with simulation capabilities and additional functionalities to streamline the development process.

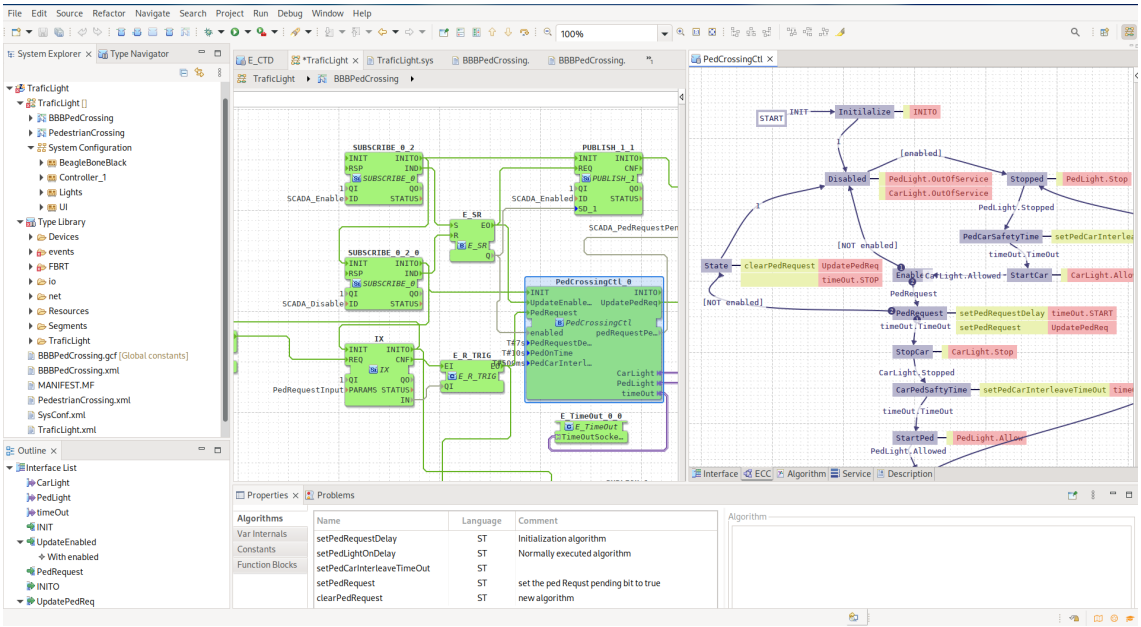

Figure 1.2: 4diac IDE [1]

1.2 Motivation

The evolution of CPPS has increased the demand for flexible and intelligent control solutions capable of integrating physical processes with digital technologies. FBD and the *IEC 61499* standard offer a structured, modular approach to building such distributed automation systems. To fully leverage these benefits, development tools must evolve alongside industrial needs, supporting more accessible, collaborative, and scalable workflows.

1.3 Problem

Existing *IEC 61499* development environments lack modern capabilities such as remote access, real-time collaboration, and seamless integration with cloud infrastructures. These limitations hinder efficient development, maintenance, and deployment of control applications within distributed CPPS. As a result, current tools fall short in supporting the flexibility and scalability required by certain modern industrial automation systems. Besides this, the complexity of these current IDEs rules them out for many simpler use cases where the time wasted on setting up and learning them is not justifiable.

1.4 Research Questions

This research seeks to investigate how modern web and distribution technologies can be leveraged to enhance the development, deployment, and operation of *IEC 61499*-compliant systems. This study is guided by the following main research question:

> How can we design IDEs that improve the development, deployment, and collaboration processes for *IEC 61499* applications in distributed CPPS?

To further refine the scope, the following sub-question is proposed:

> What architectural and technological requirements must be considered when designing an IDE for *IEC 61499*?

Chapter 2

Literature Review

This chapter reviews prior work related to *IEC 61499*-based development and the used IDEs. The goal is to better understand the landscape, which will then allow the critical review of the state of the art.

2.1 Methodology

A targeted literature review was carried out, focusing on studies addressing *IEC 61499* development tools, function blocks, CPPS, and development environments, to then critically reflect upon their findings to help the writing of this dissertation and the planning of the final product.

2.1.1 Search and Selection

The search was conducted in digital libraries *ScienceDirect*, *IEEE Xplore*, *ACM Digital Library*, among others, as well as research paper aggregators like *Semantic Scholar*, *Scopus* and *Web of Science* using combinations of the following terms:

- *IEC 61499*
- FBD
- IDE
- CPPS

Following the search, a manual review process was conducted to refine and validate the results. Titles and abstracts were first screened to understand the relevance of the articles with respect to this dissertation. The publications were then examined in greater detail through full-text review, the reference lists of selected papers were manually inspected to identify relevant works that may not have been captured by the initial keyword search.

Furthermore, terms related to the state of the art that were discovered mid search were also included in some queries. The objectives of doing this were to get a better personal understanding

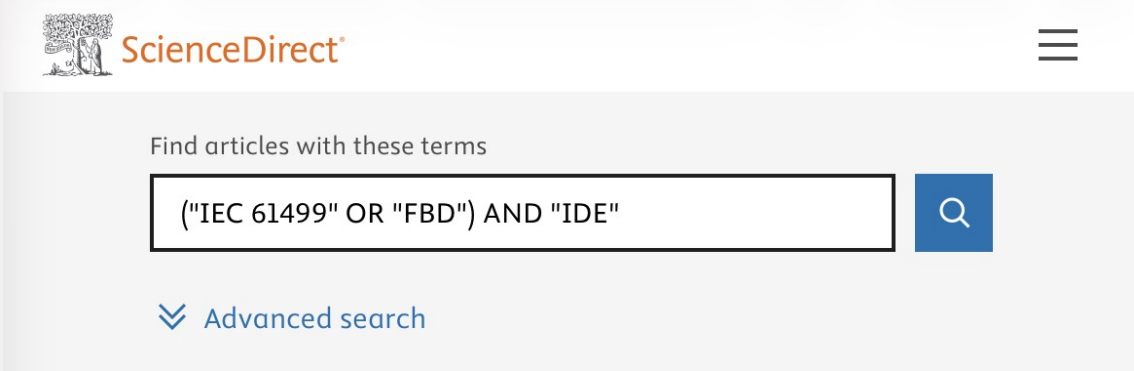

Figure 2.1: Example query on *ScienceDirect* with 136 results [10]

of the scene, as well as to find interesting references that could be included in this dissertation to backup the written claims. Some of the terms used were:

- 4diac
- collaboration

2.2 Related Work

It should be interesting to mention some papers that were used to get the knowledge used for writing this dissertation and which will be referenced throughout the whole document.

Monostori [6] writes in-depth about CPPS, while Torres, Gonçalves, and Pinto [12] discuss how *IEC 61499* can be leveraged for CPPS development. Zoitl et al. [16] demonstrated the use of the open-source *4DIAC* environment to develop modular and reusable *IEC 61499* applications, highlighting the benefits of flexibility and adaptability in industrial settings. Similarly, Pinto et al. [8] presented an industrial case study showing that function blocks oriented approaches improve system modularity and reduce development effort compared to traditional procedural programming. Some more recent studies explore web-based execution of FBD, Rohat and Popescu [9] as well as Lyu et al. [5] propose the creation of a web platform for *IEC 61499* development.

Chapter 3

State of the Art

This chapter provides a comprehensive analysis of the current landscape of development environments for CPPS and the *IEC 61499* standard. It categorises and critically examines the existing development tools, while looking for possible gaps in the ecosystem.

3.1 Ecosystem Summary

Currently, the development ecosystem is composed only by desktop-based IDEs. These tools are typically monolithic applications installed locally on engineering workstations.

3.1.1 Open-Source

The most prominent open-source implementation is the *Eclipse 4diac* IDE (based on the *Eclipse* framework) which supports a variety of RTEs such as *4diac FORTE* and *DINASORE*. This IDE is very feature rich at the cost of significant system resources and increased complexity, resulting in a steep learning curve, which makes it more difficult to use [14].

An alternative is *Jetbrains FBME* which is a much simpler modelling environment [11]. While easier to use than 4diac IDE, it still is a local-first development environment. The importance of this will be explained in Section 3.2.

Figure 3.1: *4diac* logo

3.1.2 Commercial

The commercial side is occupied by *EcoStruxure Automation Expert*, *ISaGRAF Workbench* and other development environments that usually have their own built-in proprietary RTE. This makes them often very complete production ready tools, but they are not very flexible and may not be the best fit for many needs.

3.2 Local-first development environments

3.3 Evolution of Web-Based Engineering Tools

The software engineering domain has seen a massive shift from desktop IDEs to cloud-native environments. This trend is now influencing the domain of Industrial Control Systems (ICS), and while recent studies have proposed a Service-Oriented Architectures (SOA) to integrate *IEC 61499* with cloud layers [12], the engineering tools themselves remain largely desktop-bound.

3.3.1 Web-Based IDEs in Software Development

In general software development, tools like *Visual Studio Code* have started offering web-based counterparts (as seen in Figure 3.2) to their desktop applications [13] in search of a more flexible user experience. In fact, newer development environments like *Zed* make it a key goal of their products to have web platform support [15].

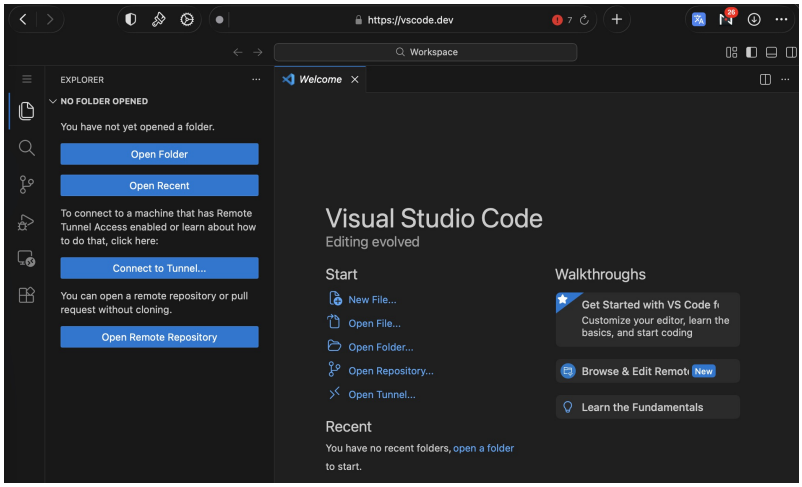

Figure 3.2: *VSCode Web* interface

3.4 Gap Analysis

Current *IEC 61499* tools face significant limitations that hinder modern CPPS development:

- **Platform Dependency:** Existing solutions are monolithic desktop applications, lacking the accessibility and portability of web-based environments.
- **Resource Intensity:** Current options are heavyweight and complex, creating a high barrier to entry compared modern development tools of other purposes.
- **Web Trends:** Despite industry trends, there is currently no mature, lightweight web interface for FBD, leaving ICS engineering behind the curve of cloud-native software development.

Chapter 4

Proposed Solution

To address the limitations identified in existing *IEC 61499* development environments, this dissertation proposes the design and implementation of a lightweight, web-based IDE for modelling *IEC 61499*-compliant applications. This work seeks to bridge the gap between modern software engineering practices and the current state of industrial automation tooling for CPPS. A web-based IDE solves many of the problems mentioned in Chapter 3, addressing the identified gap.

The main reason to propose using web technologies is to solve the issues that affect any local-first development environment. By having the IDE be hosted by a centralised server, the following is achieved:

- **Ease of setup**. Instead of each team member having to install a resource heavy program and set it up with the team's code and configurations, a single server is set up and the team members simply have to connect to it via their browser.
- **Ease of deployment**. Only the server needs to be aware of all of the target RTEs and have connection to them. When a team member wants to deploy, the request will be routed through the server, acting as the middleware, removing the need for each development machine to have connection with the target RTEs.
- **Real-time collaboration**. All data is kept and directly modified in the server itself, instead of requiring a periodic synchronisation system like *git*. FBD coded programs, since they are meant for visual editing, need to be stored in formats that don't easily handle merge conflicts. Real-time combats this by keeping track of atomic changes, instead of blocks of text.

4.1 Approach

The approach begins with the derivation of functional and non-functional requirements from the *IEC 61499* standard and an analysis of existing desktop-based tools. Core requirements include support for function block types, event and data connections, and deployment to compliant

runtime environments, namely *DINASORE*. Non-functional requirements emphasise platform independence, reduced complexity, usability, and extensibility, addressing the resource intensity and accessibility issues of current solutions. The use of modern web technologies further allows the inclusion of collaborative features and scalable workflows, which are largely absent from current *IEC 61499* tools.

The effectiveness of the approach is assessed through the implementation of a functional prototype and its direct comparison with established desktop-based IDEs like *4diac*, focusing on usability, accessibility, and workflow efficiency. This evaluation demonstrates the feasibility of applying web-based principles to *IEC 61499* development and their potential benefits for distributed CPPS.

4.2 Methods

This section describes the research methodology adopted to address the research questions defined in this dissertation. Given the practical and design-oriented nature of the problem, the methodology focuses on the design, implementation, and evaluation of a piece of software. The chosen methods are appropriate for investigating how web-based technologies can enhance *IEC 61499* development environments, as they allow both the realisation of a concrete solution and the systematic assessment of its capabilities relative to existing tools.

4.2.1 Research Design

This research follows a qualitative, design-oriented research approach. The core of the research consists of the design and implementation of a functional prototype of the solution. Rather than aiming for statistical generalisation, the study focuses on demonstrating feasibility and identifying qualitative improvements over existing desktop-based environments. The prototype serves as a research artifact used to explore architectural decisions, usability considerations, and workflow implications.

4.2.2 Evaluation

Evaluation is conducted through qualitative analysis and comparative assessment against established *IEC 61499* IDEs. Criteria such as accessibility, usability, feature coverage, and support for collaborative workflows are used to assess how effectively the proposed solution addresses the limitations identified in the literature and state of the art.

4.3 Workplan

Figure 4.1 outlines the planned work for this dissertation. The schedule is organised into three main phases: Experimentation, Code Development, and Deliveries. During the Experimentation phase, there will be a focus on studying the standard and running experiments to establish a solid

technical foundation. The Code Development phase overlaps partially with Experimentation, ensuring that the software evolves in parallel with the experimental insights. Finally, the Deliveries phase spans the entire project duration and includes writing the dissertation and preparing the final presentation, ensuring that all outputs are documented and communicated effectively.

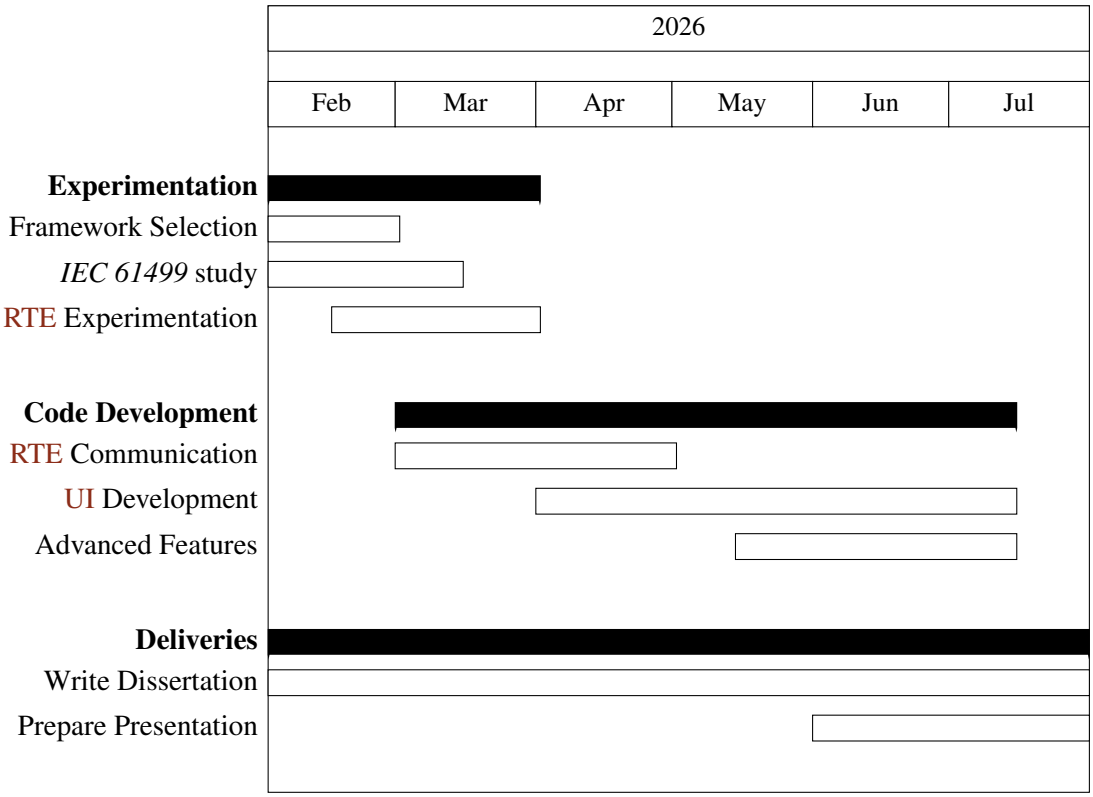

Figure 4.1: Gantt chart timeline for the proposed work

Chapter 5

Prototype Development

To be able to prove the proposed solution is viable and will successfully address the gap identified, a prototype needs to be developed and afterwards validated.

5.1 Runtime Environment

Out of all RTEs compatible with *IEC 61499*, this prototype will focus on supporting *DINASORE* (Dynamic INtelligent Architecture for Software and MOdular REconfiguration). It is much less complex than others, making it ideal for a proof of concept IDE. As an open-source project, *DINASORE* can be freely inspected and studied, which is particularly valuable in a research and prototyping context where flexibility is prioritised over real-time performance guarantees. This ease of study rules out the use of commercial RTEs like *EcoStruxure Automation Expert's "dPAC"* and *ISaGRAF Runtime*. Other open-source RTEs were considered, but none was as well-suited as *DINASORE*. *4diac FORTE* for example was way too extensive and complicated to be able to easily study it's source code.

By default, *DINASORE* uses the port *61499* to receive commands from the IDE and send responses. This is important to note as it is the port that will be used as example throughout this document. The exact details of the communication implementation will be explained further in Section 5.5.

Figure 5.1: *DINASORE* logo

5.2 Server

The design begins by setting up a server that will serve the IDE interface as a webpage, store and manage all the data and handle communication with the RTEs. Taking into account the features we plan on implementing, *JavaScript* running on *Node.js* were chosen due to the extensive library ecosystem. For real-time collaboration we will need a library to run in both the server and the frontend, since *JavaScript* runs natively in the browser, it further reinforces that it should also be used in the server. The library choice will be explained in depth in Section 5.4, proving the previous train of thought.

Being the middleware, there are 4 different types of network communications that the server will need to handle, each with it's own requirements:

- **Serve webpage**: Communicates via *GET* requests to deliver the *html*, *css* and *js* files that make up the IDE interface.
- **Real-Time Collaboration**: Constant atomic updates via a library.
- **Requested deployment**: A team member sends a *POST* request, that tells the server to deploy.
- **Deployment sockets**: Communicates via sockets with each RTE to send commands and receive the corresponding responses.

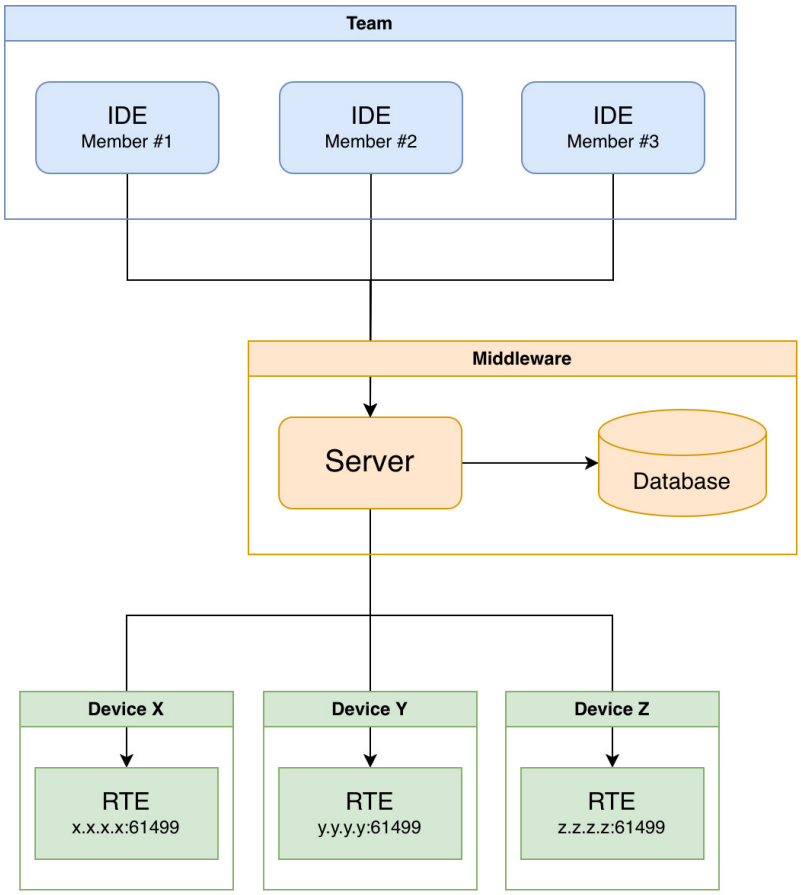

Figure 5.2: Communication through the Middleware

5.3 Frontend

The frontend is the UI that will be used to create and modify the FBD programs. It consists of a 3 views.

5.3.1 Devices Configuration View

In the interest of distributed CPPS, *IEC 61499* allows function blocks to be mapped to different *DINASOREs*, so we need a clean way to define the resources that are available.

Each *DINASORE* should have three fields that the team can configure:

- **Name**: Human friendly name to quickly identify what device this is
- **Endpoint**: The IP address of the device and the port where *DINASORE* is running
- **Color**: The color associated with this *DINASORE* instance, providing a visual distinction in the *Graph View* between function blocks mapped to different instances.

where other Local-first

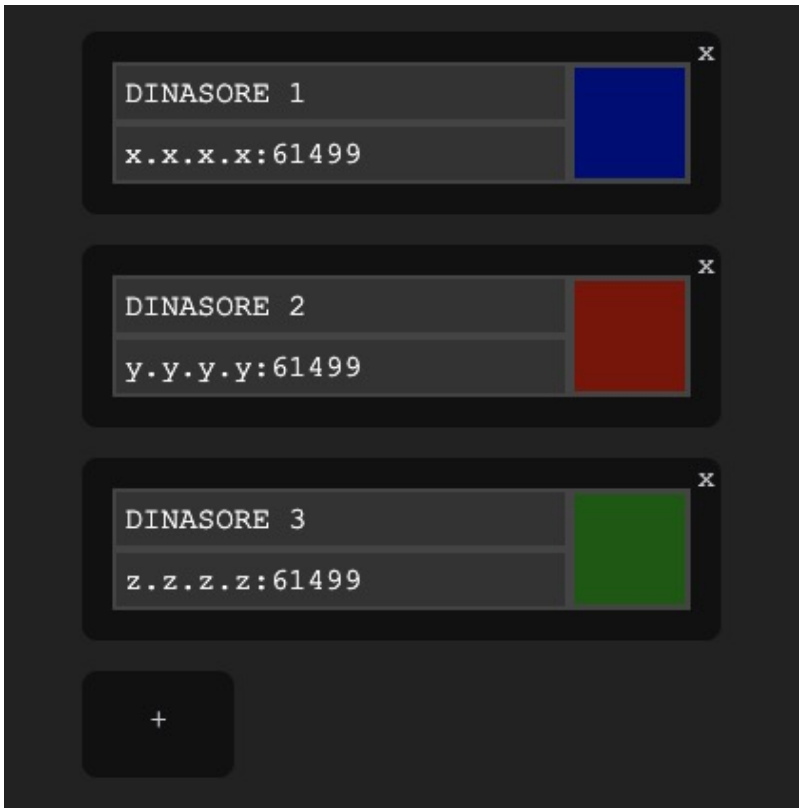

Figure 5.3: Devices Configuration View

5.3.2 Function Blocks Editor View

5.3.3 Graph View

5.4 Real-Time Collaboration

5.5 Deployment

5.5.1 Deployment Console

Chapter 6

Results and Analysis

Chapter 7

Conclusions

References

[1] *4diac IDE Website*. URL: https://eclipse.dev/4diac/4diac_ide/.

[2] Young-Jo Cho et al. "Function Block Diagram Approach for Manufacturing Process Control Programming". en. In: *IFAC Proceedings Volumes* 26.2 (July 1993), pp. 461–464. ISSN: 14746670. DOI: 10.1016/S1474-6670(17)48510-7. URL: https://linkinghub.elsevier.com/retrieve/pii/S1474667017485107 (visited on 01/28/2026).

[3] Marcelo V. Garcia et al. "Engineering tool to develop CPPS based on IEC-61499 and OPC UA for oil&gas process". en. In: *2017 IEEE 13th International Workshop on Factory Communication Systems (WFCS)*. Trondheim, Norway: IEEE, May 2017, pp. 1–9. ISBN: 978-1-5090-5788-7. DOI: 10.1109/WFCS.2017.7991969. URL: http://ieeexplore.ieee.org/document/7991969/ (visited on 01/30/2026).

[4] *IEC 61499 Wikipedia*. URL: https://en.wikipedia.org/wiki/IEC_61499.

[5] Tuojian Lyu et al. "Towards Web Platform for Cloud-Based Virtual Commissioning of IEC 61499 Distributed Automation Systems". en. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. Padova, Italy: IEEE, Sept. 2024, pp. 1–4. ISBN: 979-8-3503-6123-0. DOI: 10.1109/ETFA61755.2024.10710351. URL: https://ieeexplore.ieee.org/document/10710351/ (visited on 12/17/2025).

[6] László Monostori. "Cyber-physical Production Systems: Roots, Expectations and R&D Challenges". en. In: *Procedia CIRP* 17 (2014), pp. 9–13. ISSN: 22128271. DOI: 10.1016/j.procir.2014.03.115. URL: https://linkinghub.elsevier.com/retrieve/pii/S2212827114003497 (visited on 01/28/2026).

[7] Linna Pang et al. "Formal verification of function blocks applied to IEC 61131-3". en. In: *Science of Computer Programming* 113 (Dec. 2015), pp. 149–190. ISSN: 01676423. DOI: 10.1016/j.scico.2015.10.005. URL: https://linkinghub.elsevier.com/retrieve/pii/S0167642315002981 (visited on 01/29/2026).

[8] Gonçalo Pinto et al. "Digital Factory: An Industrial Case Study for Function Block-Oriented Development". en. In: *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. Aveiro, Portugal: IEEE, Oct. 2023, pp. 01–06. ISBN: 979-8-3503-1161-7. DOI: 10.1109/WF-IoT58464.2023.10539486. URL: https://ieeexplore.ieee.org/document/10539486/ (visited on 01/29/2026).

[9] Oana Rohat and Dan Popescu. "Remote web-based execution of IEC 61499 function blocks". en. In: *Proceedings of the 2014 6th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*. Bucharest, Romania: IEEE, Oct. 2014, pp. 33–38. ISBN: 978-1-4799-5478-0 978-1-4799-5479-7. DOI: 10.1109/ECAI.2014.7090220. URL: http://ieeexplore.ieee.org/document/7090220/ (visited on 01/28/2026).

[10] *ScienceDirect*. URL: https://www.sciencedirect.com.

[11] Radimir Sorokin, Sandeep Patil, and Valeriy Vyatkin. "Novel development tool for IEC 61499 based on domain-specific languages". en. In: *IFAC-PapersOnLine* 55.2 (2022), pp. 439–444. ISSN: 24058963. DOI: 10.1016/j.ifacol.2022.04.233. URL: https://linkinghub.elsevier.com/retrieve/pii/S2405896322002348 (visited on 01/28/2026).

[12] Tomás Torres, Gil Gonçalves, and Rui Pinto. "Literature Review on Cloud-Based Service-Oriented Architecture for IEC 61499 Distributed Control Systems:" en. In: *Proceedings of the 15th International Conference on Cloud Computing and Services Science*. Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 174–181. ISBN: 978-989-758-747-4. DOI: 10.5220/0013293400003950. URL: https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0013293400003950 (visited on 01/28/2026).

[13] *VSCode Web*. URL: https://code.visualstudio.com/docs/setup/vscode-web.

[14] Thomas Weber, Alois Zoitl, and Heinrich HuBmann. "Usability of Development Tools: A CASE-Study". en. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. Munich, Germany: IEEE, Sept. 2019, pp. 228–235. ISBN: 978-1-7281-5125-0. DOI: 10.1109/MODELS-C.2019.00037. URL: https://ieeexplore.ieee.org/document/8904489/ (visited on 01/30/2026).

[15] *Zed Roadmap*. URL: https://zed.dev/roadmap.

[16] Alois Zoitl, Thomas Strasser, and Gerhard Ebenhofer. "Developing modular reusable IEC 61499 control applications with 4DIAC". en. In: *2013 11th IEEE International Conference on Industrial Informatics (INDIN)*. Bochum, Germany: IEEE, July 2013, pp. 358–363. ISBN: 978-1-4799-0752-6. DOI: 10.1109/INDIN.2013.6622910. URL: http://ieeexplore.ieee.org/document/6622910/ (visited on 01/29/2026).